

Dylemat podwójnego agenta

- **Limit czasu:** 12 minut.
- **Przestrzeń dyskowa:** 5 GB
- **Środowisko:** 1 GPU (≈16 GB VRAM), bez dostępu do internetu
- **Rozmiar rozwiązania:** `solution.ipynb` ≤ 1 MB
- **Wynik bazowy:** 0

W krajowym centrum AI w Astanie dwa modele komputerowe — Model R (ResNet-18) i Model V (ViT-Tiny) — analizują zdjęcia. Obecnie oba modele wykonują swoje zadanie bezbłędnie, osiągając dokładność 100% i zgadzając się co do każdego obrazu. Aby sprawdzić, jak bardzo różnią się ich inteligentne „mózgi”, główny naukowiec stawia przed Tobą wyzwanie: wprowadź w każdym zdjęciu niewielkie, niemal niewidoczne zmiany pikseli, tak aby Model R i Model V zupełnie się ze sobą nie zgadzały.



1. Zadanie

Dwa wytrenowane klasyfikatory obrazów analizują ten sam obraz. Dla obrazów udostępnionych w tym zadaniu oba klasyfikatory osiągają dokładność 100%.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `vit_tiny_patch16_224` z `timm` (Transformer, ViT-Tiny).

Twoim zadaniem jest utworzenie niewielkiej zmiany („zaburzenia”, ang. *perturbation*) dla każdego obrazu, tak aby oba modele się ze sobą nie zgadzały. Dla każdego obrazu musisz utworzyć **dwa różne** zaburzenia:

- **Typ A:** po jego dodaniu Model R nadal poprawnie klasyfikuje obraz, ale Model V klasyfikuje go niepoprawnie.
- **Typ B:** po jego dodaniu Model V nadal poprawnie klasyfikuje obraz, ale Model R klasyfikuje go niepoprawnie.

Każde zaburzenie musi być na tyle *małe*, aby trudno było je zauważyć. Mniejsze zaburzenia uzyskują wyższy wynik (zob. Sekcja 5). Zaburzenie jest stosowane bezpośrednio do oryginalnego obrazu na poziomie pikseli.

2. Dane publiczne

Wraz z zadaniem udostępniono zbiór obrazów podzielony na dwa podzbiory — `train` (100 obrazów) oraz `test_public` (100 obrazów) — z których każdy zawiera obrazy o różnych rozdzielczościach. Wszystkie obrazy pochodzą z 1000 klas ImageNet-1K, a zarówno Model R, jak i Model V osiągają dokładność 100% na obu podzbiórach.

Udostępniono następujące pliki:

```
train/images/*.png      # 100 obrazów w formacie PNG
train/labels.json       # mapuje indeks każdego obrazka do jego poprawnej klasy
test_public/images/*.png # 100 obrazków w formacie PNG
test_public/labels.json  # mapuje indeks każdego obrazka do jego poprawnej klasy
```

Podczas oceniania Twój folder `dataset/test_public/` zostanie w sposób niewidoczny zastąpiony dwoma ukrytymi zbiorami obrazów (`test_leaderboard_a` i `test_leaderboard_b`) używanymi do oficjalnego oceniania. Każdy z nich zawiera **100 obrazów** w formacie PNG oraz plik z etykietami.

Uwaga: W tym zadaniu etykiety w testowych datasetach są dostępne.

3. Format danych wyjściowych

Dla każdego obrazu musisz utworzyć dwa pliki:

```
{index}_a.pt  # Zaburzenie Typu A
{index}_b.pt  # Zaburzenie Typu B
```

- `{index}` (0, 1, 2, ...), odpowiada nazwie obrazu w datasetach.
- Każdy plik zawiera pojedynczy tensor zapisany za pomocą `torch.save`. Jego kształt musi wynosić $3 \times H \times W$, gdzie H i W odpowiadają **oryginalnej** rozdzielczości tego obrazu (a nie 224×224).
- Kod powinien utworzyć tylko jeden plik ZIP, `submission.zip`. Umieść wszystkie pliki `.pt` na najwyższym poziomie archiwum ZIP, bez folderu nadrzędnego ani podkatalogów.

```
submission.zip
├── 0_a.pt
├── 0_b.pt
├── 1_a.pt
└── ...
```

Notebook ostrzeże Cię o wszelkich problemach z formatem danych wyjściowych.

4. Ograniczenia

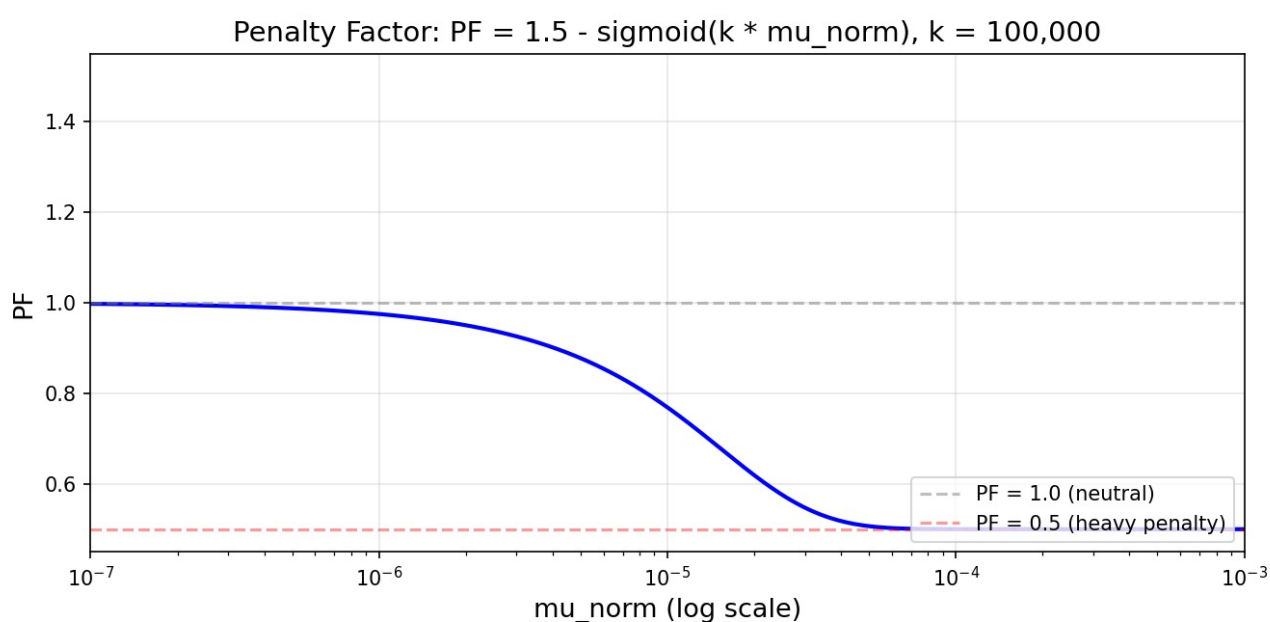
- **Modele:** Musisz użyć `torchvision.models.resnet18(pretrained=True)` i `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Żadne inne pretrenowane modele nie są dozwolone.
- **Pipeline transformacji (ang. transform pipeline); (wymuszany podczas ewaluacji):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. Zobacz Sekcję 3 pliku `baseline.ipynb` w celu uzyskania szczegółów.
- **Rozdzielczość zaburzonego obrazu:** Musi odpowiadać **oryginalnej** rozdzielczości surowego obrazu (ang. raw image resolution) (a nie 224×224). Tensor jest dodawany do surowego obrazu *przed* zastosowaniem potoku transformacji.
- **Format danych wyjściowych:** wyłącznie pliki `.pt` — bez PNG/JPG. Tensory są dodawane do obrazu (znormalizowanego do `[0, 1]`), a następnie wartości pikseli są ograniczane (ang. clipped) do `[0, 1]` przed przetwarzaniem wstępnym.
- **Nazewnictwo plików:** Płaska lista plików, ściśle według formatu `{index}_a.pt / {index}_b.pt`. Bez podkatalogów wewnątrz pliku zip.
- **Biblioteki:** `torch`, `torchvision`, `timm`.

5. Punktacja

Końcowy wynik jest obliczany w następujący sposób. Niech M oznacza liczbę obrazów w podzbiorze, $Score_A$ liczbę udanych zaburzeń Typu A, a $Score_B$ liczbę udanych zaburzeń Typu B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF jest funkcją zaprojektowaną tak, aby karać zaburzenia o wysokiej normie i być bardzo czułą w pobliżu górnej granicy wyników. Jest ona ograniczona do zakresu od 0.5 do 1. Pełną implementację można zobaczyć w Sekcji 8 pliku `solution.ipynb`.



Rysunek: Krzywa funkcji kary.

6. Sprawdzanie zgłoszenia

W Sekcji 7 notebooka `solution.ipynb` znajdują się testy, które ostrzegą Cię o problemach z formatowaniem.

7. Testowanie lokalne

`solution.ipynb` zawiera kompletny, działający przykład. Wczytuje on dane publiczne, oba modele i oficjalny moduł oceniający (ang. `scorer`), a następnie zapisuje plik ZIP ze zgłoszeniem. Przeczytaj go przed rozpoczęciem pracy.

8. Sposób zgłoszenia

- Zapisz swoje zmiany w `solution.ipynb`.
- Otwórz kartę Git na lewym pasku bocznym JupyterLab.
- Wykonaj **Stage** dla `solution.ipynb` (ikona + obok niego).
- Wprowadź komunikat commita i kliknij **Commit**.
- Kliknij ikonę chmury ze strzałką w górę, aby wykonać push.
- Wróć na stronę tego konkursu i kliknij **Submit**.

Prześlij dokładnie jeden plik o nazwie `solution.ipynb`.